

Day 1 Revision Sheet

Prefix Sum & Difference Array

৫টি সমাধান করা প্রবলেম, লাইন বাই লাইন ব্যাখ্যা সহ (বাংলা + English)

BUBT Intra Contest Prep — 13 Day Roadmap

Your own practice file — prefix sum, difference array, and XOR-difference variant

Idea: Three back-to-back demos of the same "build an array, then answer range queries fast" idea. Prefix sum handles range-**sum**. Difference array handles range-**update**. The XOR version swaps every + for ^ — proof that the pattern isn't really about addition, it's about "an operation with an inverse applied over a range."

Cheat sheet match: Page 7 "PREFIX SUM (1D)" and Page 9 "DIFFERENCE ARRAY" — your `diff[l]+=x; diff[r+1]-=x;` line is copied almost exactly from the sheet.

YOUR CODE (ANNOTATED)

```
// prefixsum: pre[i] = sum of arr[0..i-1]
void prefixsum(const vector<ll> &arr, int n) {
    vector<ll> pre(n + 1, 0); // size n+1, pre[0]=0 always
    for (int i = 0; i < n; i++)
        pre[i + 1] = pre[i] + arr[i]; // running total, shifted by 1
    // rangeSum(l,r) = pre[r+1] - pre[l] -- O(1) after this
}

// differencearray: apply +x to arr[l..r] in O(1) instead of O(r-l)
void differencearray(vector<ll> &arr, int n, int l, int r, ll x) {
    vector<ll> diff(n + 1, 0);
    diff[l] += x; // start adding x from index l
    if (r + 1 < n) diff[r + 1] -= x; // cancel it out right after r

    ll curr = 0;
    for (int i = 0; i < n; i++) {
        curr += diff[i]; // curr = "how much extra applies here"
        arr[i] += curr; // reconstruct real array from diff
    }
}

// xorversion: SAME shape, but XOR is its own inverse (x^x=0)
void xorversion(vector<ll> &arr, int n, int l, int r, ll x) {
    vector<ll> diff(n + 1, 0);
    diff[l] ^= x;
    if (r + 1 < n) diff[r + 1] ^= x; // XOR again = undo (a^x^x=a)

    ll curr = 0;
    for (int i = 0; i < n; i++) {
        curr ^= diff[i];
        arr[i] ^= curr;
    }
}
```

WHY IT WORKS — 3 LINES

- 1. Prefix sum turns "sum of a range" into `pre[r+1]-pre[l]` — one subtraction instead of a loop.
- 2. Difference array is prefix sum **backwards**: instead of reading fast, you *write* fast (range update), then do one pass to reconstruct.
- 3. The XOR version proves the difference-array trick generalizes to any operation that has an inverse (+ / -, or ^ / ^).

বাংলা ব্যাখ্যা

- 1. প্রিফিক্স সাম দিয়ে যেকোনো রেঞ্জের সাম বের করতে পুরো লুপ চালাতে হয় না — শুধু `pre[r+1]-pre[l]` করলেই হয়ে যায়।
- 2. ডিফারেন্স অ্যারে হলো প্রিফিক্স সামের উল্টো — আগে থেকে পুরো রেঞ্জ আপডেট করতে চাইলে, প্রতিটা ইনডেক্সে গিয়ে আপডেট না করে শুধু দুই জায়গায় (শুরু আর শেষের পরের ঘরে) পরিবর্তন করলেই চলে, পরে এক পাসে রিকনস্ট্রাক্ট করা হয়।
- 3. XOR ভার্সনটা দেখায় যে এই ট্রিকটা শুধু যোগ-বিয়োগের জন্য না — যেকোনো এমন অপারেশনের জন্য কাজ করে যেটার একটা "উল্টানো" সংস্করণ আছে (যেমন XOR নিজেই নিজের উল্টো)।

1. Retrieve the Array (CodeChef ARRAYRET)

codechef.com/practice/course/1-star-difficulty-problems/DIFF1200/problems/ARRAYRET

Idea: You're given array `b`, which was built by repeatedly applying a "circular difference array" style operation to some hidden array `a`. The key insight: summing all of `b` and dividing by $(n+1)$ recovers the constant amount that was added everywhere — from there each $a[i] = b[i] - \text{sum_a}$ falls straight out. It's less "code the algorithm" and more "reverse-engineer the math behind the difference-array process."

Cheat sheet match: Page 9 "DIFFERENCE ARRAY" — this problem is the *reverse* direction of the pattern: given the final array, recover the original, instead of given the original, build the final array.

YOUR CODE (ANNOTATED)

```
void solve() {
    int n; cin >> n;
    vector<ll> b(n);
    ll sum_b = 0;
    for (int i = 0; i < n; i++) {
        cin >> b[i];
        sum_b += b[i];           // accumulate() pattern from cheat sheet, done manually
    }

    // KEY STEP: total "extra" spread across n+1 slots -> divide it back out
    ll sum_a = sum_b / (n + 1);

    for (int i = 0; i < n; i++)
        cout << b[i] - sum_a << " "; // undo the shift, one subtraction each
}
```

WHY IT WORKS — 3 LINES

1. The construction process adds the same total mass `sum_a` across `n+1` "slots" — so `sum_b = sum_a * (n+1)` .
2. That means `sum_a` is just `sum_b / (n+1)` — pure math, no loop needed to find it.
3. Once you know `sum_a` , every `a[i]` is recovered independently — $O(n)$, no reconstruction pass needed like a normal difference array.

বাংলা ব্যাখ্যা

1. মূল অ্যারে `a` থেকে `b` বানানোর প্রসেসে একই পরিমাণ ভ্যালু (`sum_a`) মোট `n+1` বার যোগ হয়েছে — তাই `sum_b = sum_a * (n+1)` ।
2. এখান থেকে `sum_a` বের করতে কোনো লুপ লাগে না — শুধু `sum_b` কে `(n+1)` দিয়ে ভাগ করলেই হয়ে যায়।
3. `sum_a` জানা হয়ে গেলে প্রতিটা `a[i]` আলাদাভাবে বের করা যায় — সাধারণ ডিফারেন্স অ্যারের মতো আলাদা রিকনস্ট্রাকশন পাসের দরকার হয় না, একটাই লুপে কাজ শেষ।

codeforces.com/problemset/problem/295/A

Idea: This is difference array applied **twice, stacked**. You have m range-add operations, but each operation itself might be applied multiple times (specified by k "meta-operations" that are themselves ranges over the operation list). So: difference array #1 counts how many times each operation index is used $\rightarrow$ prefix sum to unroll it $\rightarrow$ difference array #2 applies each operation's real range-add to the array $\rightarrow$ prefix sum to unroll that too. Two layers of the exact same trick.

Cheat sheet match: Page 9 "DIFFERENCE ARRAY" applied twice in a row — once on the `op_count` array (how many times each operation fires), once on the actual data array `a`.

YOUR CODE (ANNOTATED)

```
void solve() {
    int n, m, k; cin >> n >> m >> k;
    vector<ll> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<int> l(m), r(m), d(m);    // the m defined operations (range + delta)
    for (int i = 0; i < m; i++) cin >> l[i] >> r[i] >> d[i];

    // LAYER 1: difference array over the OPERATION INDICES (not the data!)
    vector<ll> op_count(m + 1, 0);
    for (int i = 0; i < k; i++) {
        int x, y; cin >> x >> y;
        op_count[x - 1]++;          // diff[l] += 1 (using operations x..y)
        op_count[y]--;              // diff[r+1] -= 1
    }
    for (int i = 1; i < m; i++)
        op_count[i] += op_count[i - 1];    // prefix sum -> op_count[i] = times op i fires

    // LAYER 2: difference array over the DATA ARRAY, using op_count as the multiplier
    vector<ll> diff(n + 1, 0);
    for (int i = 0; i < m; i++) {
        ll new_d = 1LL * d[i] * op_count[i];    // total delta this op contributes
        diff[l[i] - 1] += new_d;
        diff[r[i]] -= new_d;
    }

    ll cur = 0;
    for (int i = 0; i < n; i++) {
        cur += diff[i];          // unroll layer 2
        a[i] += cur;
    }
}
```

WHY IT WORKS — 3 LINES

- 1. Naively applying k meta-operations $\times$ m range-updates $\times$ n array size would be far too slow — so count **how many times** each operation fires first, using a difference array on the operation list itself.
- 2. Once `op_count[i]` is known (via one prefix-sum unroll), each operation's *total* contribution is just `d[i] * op_count[i]` — a single value, no matter how many times it repeats.
- 3. Apply all m total contributions as one more difference array on the real data array, unroll once more — done in $O(n+m+k)$ total.

বাংলা ব্যাখ্যা

- 1. সরাসরি k টা মেটা-অপারেশন, m টা রেঞ্জ-আপডেট, আর n সাইজের অ্যারেতে একসাথে অ্যাপ্লাই করলে অনেক স্লো হয়ে যাবে — তাই প্রথমে বের করতে হবে প্রতিটা অপারেশন কতবার ব্যবহার হচ্ছে, সেটার জন্য অপারেশন লিস্টের উপর একটা ডিফারেন্স অ্যারে বসানো হয়েছে।
- 2. `op_count[i]` (এক প্রিফিক্স সাম করে বের করা) জানা হয়ে গেলে, প্রতিটা অপারেশনের মোট প্রভাব হলো শুধু `d[i] * op_count[i]` — যতবারই রিপিট হোক না কেন, একটা সংখ্যাতেই সব হিসাব হয়ে যায়।
- 3. এই মোট প্রভাবগুলো আরেকবার ডিফারেন্স অ্যারে হিসেবে আসল ডেটা অ্যারেতে বসিয়ে দিলেই কাজ শেষ — পুরো প্রসেসটা মাত্র $O(n+m+k)$ সময়ে হয়ে যায়।

3. Range Sum Query 2D — Immutable (LeetCode)

leetcode.com/problems/range-sum-query-2d-immutable

Idea: Same idea as 1D prefix sum, extended to a grid using inclusion-exclusion: to get the sum of a rectangle, take the big rectangle from the origin, then subtract the two overlapping strips above and to the left, then add back the corner you subtracted twice.

Cheat sheet match: Page 9 "PREFIX SUM (2D)" — your code is a near-exact copy of the sheet's $pre[i+1][j+1] = grid[i][j] + pre[i][j+1] + pre[i+1][j] - pre[i][j]$ formula and the rectangle-query formula right below it.

YOUR CODE (ANNOTATED)

```
class NumMatrix {
public:
    vector<vector<int>> pre;    // pre[i][j] = sum of rectangle (0,0) to (i-1,j-1)

    NumMatrix(vector<vector<int>> &matrix) {
        int n = matrix.size(), m = matrix[0].size();
        pre = vector<vector<int>>(n + 1, vector<int>(m + 1, 0));

        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                // current cell + strip above + strip left - double-counted corner
                pre[i+1][j+1] = matrix[i][j] + pre[i][j+1] + pre[i+1][j] - pre[i][j];
    }

    int sumRegion(int row1, int col1, int row2, int col2) {
        // big rect - top strip - left strip + corner (added back, was subtracted twice)
        return pre[row2+1][col2+1] - pre[row1][col2+1] - pre[row2+1][col1] + pre[row1][col1];
    }
};
```

WHY IT WORKS — 3 LINES

- 1. Build once: `pre[i][j]` stores the sum of everything from the top-left corner to `(i-1,j-1)` — costs $O(n*m)$, done only once in the constructor.
- 2. Each query then uses inclusion-exclusion on 4 corner lookups — no loop, $O(1)$ per query no matter how big the rectangle is.
- 3. Exactly the 1D prefix sum idea, just one dimension higher — subtract what you overcounted, add back what you double-subtracted.

বাংলা ব্যাখ্যা

- 1. একবারই বানাতে হয়: `pre[i][j]` তে টপ-লেফট কর্নার থেকে `(i-1,j-1)` পর্যন্ত পুরো অংশের যোগফল জমা থাকে — এটা কনস্ট্রাক্টরে মাত্র একবার $O(n*m)$ সময়ে করা হয়।
- 2. এরপর প্রতিটা কোয়েরিতে মাত্র ৪টা কর্নার ভ্যালু দিয়ে ইনক্লুশন-এক্সক্লুশন করলেই উত্তর পাওয়া যায় — কোনো লুপ লাগে না, রেক্ট্যাঙ্গেল যত বড়ই হোক $O(1)$ সময়ে হয়ে যায়।
- 3. এটা আসলে ১ডি প্রিফিক্স সামেরই একটা এক্সটেনশন, শুধু এক ডাইমেনশন বেশি — যেটা বেশি গোনা হয়েছে সেটা বাদ দাও, যেটা দুইবার বাদ দেওয়া হয়েছে সেটা আবার যোগ করে দাও।

4. Prefix Sum Addicts (CF 1738B)

PREFIX SUM + CONSTRUCTIVE

codeforces.com/problemset/problem/1738/B

Idea: You're only given the **last k** prefix sums of a sorted, non-decreasing sequence, and need to decide if a valid full sequence could exist. Recover the last k-1 actual array elements via $a[i] = pre[i] - pre[i-1]$ (prefix sum in reverse), check they respect the non-decreasing property, then verify the remaining unknown prefix ($pre[n-k+1]$) is achievable given the smallest known element as an upper bound per unknown slot.

Cheat sheet match: Page 7 "PREFIX SUM (1D)" — but run **backwards**: $a[i] = pre[i] - pre[i-1]$ is literally solving $pre[i+1] = pre[i] + a[i]$ for $a[i]$ instead of for pre .

YOUR CODE (ANNOTATED)

```
void solve() {
    int n, k; cin >> n >> k;
    vector<ll> pre(n + 1);
    for (int i = n - k + 1; i <= n; i++) cin >> pre[i]; // only last k prefix sums given

    if (k == 1) { cout << "Yes\n"; return; } // trivial: single value always valid

    vector<ll> a(n + 1);
    // REVERSE prefix sum: recover actual elements from given prefix sums
    for (int i = n - k + 2; i <= n; i++)
        a[i] = pre[i] - pre[i - 1];

    // check the recovered elements are non-decreasing (sorted array requirement)
    for (int i = n - k + 2; i < n; i++)
        if (a[i] > a[i + 1]) { cout << "No\n"; return; }

    // feasibility check: can the unknown prefix pre[n-k+1] be split into
    // (n-k+1) values, each <= smallest known element a[n-k+2]?
    if (pre[n-k+1] <= a[n-k+2] * (n-k+1))
        cout << "Yes\n";
    else
        cout << "No\n";
}
```

WHY IT WORKS — 3 LINES

- Prefix sum differences ($pre[i] - pre[i-1]$) recover the exact array values you weren't given directly.
- A valid sorted array means every recovered value must be $\leq$ the next one — one linear scan checks this.
- The unknown prefix is only feasible if it can be "hidden" using values no larger than the smallest known one — checked with one multiplication, no search needed.

বাংলা ব্যাখ্যা

- প্রিফিক্স সামের পার্থক্য ($pre[i] - pre[i-1]$) বের করলেই সরাসরি না দেওয়া আসল অ্যারের ভ্যালুগুলো পাওয়া যায়।
- অ্যারেটা সর্টেড হতে হলে প্রতিটা বের করা ভ্যালু তার পরেরটার চেয়ে ছোট বা সমান হতে হবে — এটা একটা মাত্র লুপে চেক করা যায়।
- যে অংশটা এখনও অজানা ($pre[n-k+1]$), সেটা তখনই সম্ভব যদি সেই অংশের প্রতিটা ভ্যালু জানা সবচেয়ে ছোট ভ্যালুর চেয়ে বড় না হয় — এটা একটা গুণ করেই চেক করা যায়, আলাদা সার্চের দরকার নেই।

codeforces.com/problemset/problem/1779/C

Idea: You may negate any elements to keep all prefix sums ≤ 0 . Greedy insight: process from the "pivot" outward in both directions, and whenever the running prefix sum goes positive, negate the **largest** element seen so far (to minimize the sum drop needed) — a max-heap picks that greedily on one side, a min-heap mirrors it on the other side.

Cheat sheet match: Page 7 prefix sum idea (running sum) combined with Page 12/13 "PRIORITY QUEUE (heap)" greedy-with-heap pattern — "always take the biggest offender" is the same shape as the "k largest elements" heap demo on your sheet.

YOUR CODE (ANNOTATED)

```
void solve() {
    int n, m; cin >> n >> m;
    vector<ll> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    int operations = 0;

    // LEFT SIDE (indices m-1 down to 1): keep running sum <= 0
    priority_queue<ll> max_pq; // max-heap: negate the BIGGEST value first
    ll sum = 0;
    for (int i = m - 1; i >= 1; i--) {
        sum += a[i];
        if (a[i] > 0) max_pq.push(a[i]);
        while (sum > 0) { // sum too high -> must negate something
            ll x = max_pq.top(); max_pq.pop();
            sum -= 2 * x; // negating x changes sum by -2x
            operations++;
        }
    }

    // RIGHT SIDE (indices m to n-1): mirror logic on the other side
    priority_queue<ll, vector<ll>, greater<ll>> min_pq; // min-heap variant from cheat sheet
    sum = 0;
    for (int i = m; i < n; i++) {
        sum += a[i];
        if (a[i] < 0) min_pq.push(a[i]);
        while (sum < 0) {
            ll x = min_pq.top(); min_pq.pop();
            sum -= 2 * x; // x is negative here, so -2x makes sum go up
            operations++;
        }
    }

    cout << operations << endl;
}
```

WHY IT WORKS — 3 LINES

- 1. A greedy rule: whenever the running sum crosses the limit, always fix it using the **most extreme** value seen so far — that gives the biggest correction per operation, minimizing total operations.
- 2. A heap (max-heap on the left, min-heap on the right) always gives you that extreme value in $O(\log n)$, matching your cheat sheet's "k largest/smallest" heap pattern exactly.
- 3. Splitting the array at pivot m and running the mirrored greedy on each side independently keeps the whole thing linear-ish, $O(n \log n)$ total.

বাংলা ব্যাখ্যা

- 1. একটা গ্রিডি রুল: রানিং সাম যখনই লিমিট ক্রস করে, তখন সবসময় এখন পর্যন্ত দেখা সবচেয়ে **বড়** ভ্যালুটা দিয়ে ঠিক করতে হবে — এতে প্রতি অপারেশনে সবচেয়ে বেশি কারেকশন হয়, ফলে মোট অপারেশন সংখ্যা কম লাগে।
- 2. হিপ (বাম দিকে ম্যাক্স-হিপ, ডান দিকে মিন-হিপ) ব্যবহার করলে সেই এক্সট্রিম ভ্যালুটা সবসময় $O(\log n)$ সময়ে পাওয়া যায় — এটা ঠিক তোমার চিট শিটের "k largest/smallest" হিপ প্যাটার্নের মতোই।
- 3. অ্যারেটাকে m পিভটে ভাগ করে দুই দিকে আলাদাভাবে একই গ্রিডি লজিক চালালে পুরো প্রসেসটা প্রায় লিনিয়ার সময়ে, মোট $O(n \log n)$ -এ হয়ে যায়।